Módulo 3: Base de datos

> **Enunciado.** `orders` tiene 10 millones de registros y `customers` 500.000. Esta consulta es muy lenta y se ejecuta con frecuencia; se piden al menos tres estrategias de optimización.
>
> ```
> SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
> FROM orders o JOIN customers c ON o.customer_id = c.customer_id
> WHERE c.country = 'México';
> ```

Respuesta corta. Primero diagnostico con el plan real (filas estimadas frente a reales). Después aplico, en este orden: estadísticas con histograma en `country`, índices compuestos cubrientes, rediseño de la consulta (paginación por *keyset* y rango de fechas) y, si es un reporte, particionamiento o vista materializada. En la demo (2M orders / 100k customers, Oracle 23.26 Free), la paginación por *keyset* bajó la consulta de **226 ms y 42.073 lecturas a menos de 1 ms y 502 lecturas**, y el índice redujo un **86 %** el tiempo cuando el país es poco frecuente.

1. Diagnóstico primero

Optimizar sin mirar el plan es adivinar. Uso el plan **real**, no solo el estimado:

```
-- Plan estimado (no ejecuta la consulta)
EXPLAIN PLAN FOR SELECT ...;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- Plan REAL de la última ejecución, con filas estimadas frente a reales
SELECT /*+ gather_plan_statistics */ o.order_id, ... ;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

Qué mirar	Señal de problema
`TABLE ACCESS FULL` sobre `orders`	Se leen las 10M filas para devolver una fracción
E-Rows frente a A-Rows	Si difieren mucho, el optimizador decide con datos equivocados (estadísticas desactualizadas o falta de histograma)
Tipo de join	`NESTED LOOPS` es bueno para pocas filas; `HASH JOIN` lo es para muchas. Un join inadecuado suele venir de una mala estimación
`Buffers` y `Reads`	Lecturas lógicas y físicas: el costo real en bloques
Sección *Predicate Information*	Funciones sobre columnas (`UPPER(country)`) o conversiones implícitas que impiden usar índices

MySQL: `EXPLAIN ANALYZE` (8.0.18+) muestra filas estimadas y reales; `EXPLAIN FORMAT=TREE` muestra el tipo de join.

2. Estrategias

2.1 Estadísticas actualizadas e histograma en `country`

Qué resuelve. La distribución por país está sesgada. Sin histograma, Oracle supone que todos los países pesan igual (1 / número de valores distintos) y estima mal la cardinalidad, lo que lleva a elegir mal el join y el método de acceso.

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'CUSTOMERS',
    method_opt => 'FOR ALL COLUMNS SIZE 1 FOR COLUMNS country SIZE 254', cascade => TRUE);
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS', cascade => TRUE);
END;
/
```

- **Impacto medido:** las estimaciones pasaron de 10.000 clientes para **cualquier** país a 19.027 (México) y 2.039 (Costa Rica), iguales a las reales (§4.3). Sin índices el plan no cambió (no hay alternativa a los *full scans*), pero es el **requisito** para que las demás estrategias se elijan bien.
- **Costo:** bajo; debe programarse (el job automático de estadísticas de Oracle lo hace en ventanas nocturnas).
- *MySQL 8:* `ANALYZE TABLE customers UPDATE HISTOGRAM ON country WITH 64 BUCKETS;`

2.2 Índices compuestos y cubrientes

Qué resuelve. El filtro por país y el join por `customer_id` sin índices obligan a recorrer ambas tablas completas. Un índice **cubriente** contiene todas las columnas que la consulta necesita, así que no hace falta visitar la tabla.

```
CREATE INDEX ix_customers_pais ON customers (country, customer_id, customer_name);
CREATE INDEX ix_orders_cliente ON orders (customer_id, order_date, total_amount, order_id);
```

`order_id` se agrega al final para que el índice cubra todas las columnas del `SELECT`. Además, la FK `orders.customer_id` queda indexada (Oracle no lo hace automáticamente).

- **Impacto medido:** depende de la **selectividad** (§4.3). Con Costa Rica (2 %): de 134 ms a **19 ms** (−86 %) con *nested loops*. Con México (19 %): las lecturas bajan un 76 %, pero el tiempo no mejora (*hash join* sobre el índice).
- **Costo:** espacio (75 MB a esta escala) y escrituras más lentas en `orders` (cada inserción mantiene un índice más).
- *MySQL (InnoDB):* igual. Como el índice secundario incluye la PK implícitamente, `(customer_id, order_date, total_amount)` ya cubre `order_id`; `EXPLAIN` muestra `Using index`.

2.3 Particionamiento de `orders`

Qué resuelve. Con 10M de filas y crecimiento continuo, la mayoría de consultas operativas miran un periodo reciente. Particionar por rango de `order_date` permite el **partition pruning**: leer solo las particiones del periodo.

```
CREATE TABLE orders_part
  PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH'))
  (PARTITION p_inicial VALUES LESS THAN (DATE '2022-01-01'))
  AS SELECT * FROM orders;
CREATE INDEX ix_orders_part_cliente ON orders_part (customer_id, order_date, total_amount, order_id) LOCAL;
-- Variante compuesta: ... SUBPARTITION BY HASH (customer_id) SUBPARTITIONS 8 (paraleliza los joins por
cliente)
```

- **Impacto medido** (consulta acotada al último trimestre): de 81 ms y 9.761 lecturas a **22 ms y 758 lecturas** (−73 % / −92 %). Se leyeron 3 de 49 particiones.
- **Limitación:** si la consulta **no** filtra por fecha, el particionamiento por fecha no ayuda. Es una estrategia para combinar con 2.5.
- **Costo:** licencia (en Oracle Enterprise es una opción de pago), una migración de la tabla y disciplina en las consultas. A cambio, se pueden archivar o purgar particiones antiguas con `DROP PARTITION` en segundos.
- *MySQL:* `PARTITION BY RANGE (YEAR(order_date))…`, con la restricción de que la clave de partición debe estar en toda clave única.

2.4 Vista materializada

Qué resuelve. Si la consulta es de **reporte** (no transaccional), se puede pre-calcular el join y leer el resultado.

```
CREATE MATERIALIZED VIEW LOG ON orders WITH ROWID;
CREATE MATERIALIZED VIEW LOG ON customers WITH ROWID;
CREATE MATERIALIZED VIEW mv_ordenes_cliente
  BUILD IMMEDIATE REFRESH FAST ON DEMAND        -- o ON COMMIT; o COMPLETE programado con DBMS_SCHEDULER
  AS SELECT o.ROWID o_rid, c.ROWID c_rid, o.order_id, o.order_date, c.customer_name, o.total_amount,
c.country
     FROM orders o, customers c WHERE o.customer_id = c.customer_id;
CREATE INDEX ix_mv_pais ON mv_ordenes_cliente (country);
-- Refresco incremental:  EXEC DBMS_MVIEW.REFRESH('MV_ORDENES_CLIENTE', 'F');
```

- **Impacto medido:** México de 226 ms a **126 ms** (−44 %). El refresco FAST tras insertar una orden tomó **0,28 s** y la vista quedó `FRESH`.
- **Costo:** duplica los datos (otra copia de 2M filas), los MV logs agregan trabajo a **cada** DML en `orders` y `customers`, y los datos tienen el retraso del último refresco.
- *MySQL:* no tiene vistas materializadas nativas; se emulan con una tabla resumen mantenida por eventos (`CREATE EVENT`) o por la aplicación.

2.5 Diseño de la consulta y de los datos

Qué resuelve. La consulta devuelve **todas** las órdenes de un país: 379.645 filas a esta escala, unas 1,9 millones con 10M. Ninguna pantalla consume eso. Si se ejecuta "con frecuencia", casi seguro alimenta una vista paginada o un periodo concreto.

```
-- a) Paginación keyset (índice por fecha): cada página cuesta lo mismo, sin importar su número
CREATE INDEX ix_orders_fecha ON orders (order_date, order_id, customer_id, total_amount);

SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country_code = 'MX'
  AND (o.order_date < :ult_fecha OR (o.order_date = :ult_fecha AND o.order_id < :ult_id))  -- "después de" la
última fila vista
ORDER BY o.order_date DESC, o.order_id DESC
FETCH FIRST 50 ROWS ONLY;

-- b) Rango de fechas obligatorio para listados y reportes:  AND o.order_date >= :desde
-- c) Solo las columnas necesarias (nunca SELECT *): habilita los índices cubrientes
```

- **Impacto medido:** primera página y página 201 por *keyset* en **menos de 1 ms** (518 y 502 lecturas). La misma página 201 con `OFFSET 10000` tardó 220 ms y 9.737 lecturas, porque `OFFSET` tiene que producir y descartar las 10.000 filas anteriores.

Normalizar `country`. El texto libre produce variantes: en la demo, **1.025 clientes** guardados como `Mexico` o `MEXICO` (20.371 órdenes) quedan **fuera** del resultado de `= 'México'`. No es un problema de rendimiento, sino de **datos incorrectos**. Además, `WHERE UPPER(country) = …` impediría usar el índice.

```
ALTER TABLE customers ADD country_code CHAR(2);    -- ISO 3166-1 alfa-2, idealmente con FK a un catálogo de
países
-- migración: UPDATE customers SET country_code = <mapeo normalizado>;  luego NOT NULL + índice
(country_code, customer_id, customer_name)
```

En la demo, `country_code = 'MX'` encontró **20.052** clientes, frente a 19.027 con el texto exacto.

3. Resumen y prioridad recomendada

#	Estrategia	Cuándo usarla	Costo	Prioridad
1	**Diagnóstico + estadísticas con histograma**	Siempre, antes que nada	Bajo	**1**
2	**Diseño de la consulta** (paginación *keyset*, rango de fechas, columnas necesarias) y **normalizar** `country`	Consultas de pantallas y APIs; datos de calidad dudosa	Bajo a medio (cambio en la aplicación y migración de datos)	**2**: el mayor impacto medido
3	**Índices compuestos y cubrientes**	Filtros selectivos (pocos clientes por país) y joins por FK	Espacio y escrituras más lentas	**3**
4	**Particionamiento** por fecha (y hash por cliente)	Tablas grandes que crecen, consultas por periodo, archivado	Licencia y migración	4: cuando el volumen lo justifique
5	**Vista materializada**	Reportes que toleran datos con algo de retraso	Almacenamiento y DML más costoso	5: solo para reportes

4. Demostración con datos reales

4.1 Entorno, escala y método

Aspecto	Detalle
Motor	Oracle Database 23.26 Free (imagen `gvenzl/oracle-free:23-slim`, del `docker-compose.yml` del backend). Limitado por licencia a 2 CPU y 2 GB de RAM.
Máquina	Laptop Windows 11 con Docker Desktop
Escala	**2.000.000 orders / 100.000 customers**: un quinto del enunciado, con la misma proporción 20:1. Se genera en ~25 s y permite repetir la demo en minutos. Con `ORDERS=10000000 CUSTOMERS=500000` se corre a la escala completa.
Datos	Semilla fija (reproducible). Países: Colombia 45 %, **México 20 %** (19,0 % como "México", 1 % como "Mexico" o "MEXICO"), Perú 10 %, Chile 8 %, Ecuador 7 %, Argentina 5 %, Panamá 3 %, **Costa Rica 2 %**. Orders: cliente uniforme, fechas 2022–2025. Tamaño: `orders` 312 MB, `customers` 20 MB.
Medición	Por consulta: 1 ejecución de calentamiento + 5 medidas, **trayendo todas las filas**. Se reporta la mediana del tiempo y las lecturas lógicas (`session logical reads`). Plan real con `DISPLAY_CURSOR(…'ALLSTATS LAST')` .
Advertencia	Los datos caben en la caché (lecturas físicas ≈ 0). Los **tiempos absolutos** dependen de la máquina; las **lecturas lógicas** y la **forma del plan** son la señal comparable. Cada estrategia se aplica sobre la anterior.

4.2 Resultados (corrida del 2026-10-15)

Paso	Escenario	Plan (operación principal)	Mediana ms	Lecturas lógicas	Filas
E0 Línea base (solo PK, sin histograma)	México (19 %)	Hash join + full scan de ambas tablas	226	42.073	379.645
	Costa Rica (2 %)	Hash join + full scan de ambas tablas	134	41.740	40.959
E1 + histograma en `country`	México	Igual (estimación corregida)	231	42.073	379.645
	Costa Rica	Igual (estimación corregida)	133	41.740	40.959
E2 + índices cubrientes	México	**Hash join** + *index fast full scan*	239	**10.116**	379.645
	Costa Rica	**Nested loops** + *index range scans*	**19**	**4.413**	40.959
E3 Último trimestre, sin particionar	México, `order_date ≥ 2025-10-01`	Hash join + *index fast full scan*	81	9.761	24.143
E3 Último trimestre, particionada	México, `order_date ≥ 2025-10-01`	*Partition range iterator* (3 de 49)	**22**	**758**	24.143
E4 Vista materializada	México	Full scan de la MV (sin join)	126	19.737	379.645
	Costa Rica	Full scan de la MV	49	19.400	40.959

Paso	Escenario	Plan (operación principal)	Mediana ms	Lecturas lógicas	Filas
E5 Paginación: página 1	México, 50 filas	Índice por fecha descendente + nested loops	**< 1**	518	50
E5 Página 201 con `OFFSET 10000`	México, 50 filas	Hash join + ordenamiento de 379K filas	220	9.737	50
E5 Página 201 con *keyset*	México, 50 filas	Índice por fecha + nested loops	**< 1**	502	50

Tiempos de creación: `ix_orders_cliente` 2,2 s; tabla particionada 2,2 s + índice local 2,4 s; vista materializada 2,8 s; refresco FAST 0,28 s; `ix_orders_fecha` 5,3 s. Salida completa con todos los planes: `demo/resultados/salida-completa.txt`.

4.3 Efecto de la selectividad: nested loops frente a hash join

Sin histograma, la estimación es ciega al país. Para cualquier país, Oracle estimó 10.000 clientes (100.000 / 10 valores distintos):

	México: estimado → real	Costa Rica: estimado → real
Clientes del país (E0, sin histograma)	10.000 → 19.027 (subestima ×1,9)	10.000 → 2.039 (sobreestima ×4,9)
Filas del join (E0)	198K → 379K	198K → 41K
Con histograma (E1)	19.027 → 19.027; join 377K → 379K	2.039 → 2.039; join 40.447 → 40.959

Con índices, el optimizador cambió de estrategia según la selectividad, y en ambos casos eligió bien:

- **Costa Rica (2 %, 2.039 clientes):** `NESTED LOOPS`**.** Por cada cliente hace un *range scan* en `ix_orders_cliente` (2.039 búsquedas, unas 2 lecturas cada una). Total: **4.413 lecturas**, frente a las ~10.000 que costaría recorrer todo el índice. Por eso bajó de 134 a 19 ms.
- **México (19 %, 19.027 clientes):** `HASH JOIN`**.** Hacer 19.027 búsquedas costaría unas 40.000 lecturas. Le sale más barato recorrer **una vez** el índice cubriente completo (*fast full scan*, ~10.000 bloques), que ocupa la cuarta parte de la tabla: de 42.073 a 10.116 lecturas.
- **Punto de cruce aproximado:** estimado a partir de estas mediciones (no medido directamente), está alrededor de 10.000 / 2 ≈ **5.000 clientes, ~5 % del total**. Por debajo conviene el índice con *nested loops*; por encima, el *hash join*. Un índice no es "siempre más rápido": su beneficio depende de cuántas filas filtra.
- **Por qué México no bajó de tiempo pese al −76 % de lecturas:** con todo en caché, el costo dominante es construir el hash, cruzar y **transferir 379.645 filas**, no leer bloques. En producción, con caché fría y concurrencia, leer 4 veces menos bloques sí reduce E/S y presión sobre la caché. Pero la conclusión de fondo se mantiene: para una consulta que devuelve el 19 % de 10M de filas, la solución real es **no devolverlas todas** (§2.5), y los números de E5 lo confirman.
- **La vista materializada** (E4) también es *full scan* para Costa Rica. La vista no tenía histograma (estimó 200K filas), pero aun con una estimación correcta, leer ~41K filas dispersas por índice tocaría más bloques que el recorrido completo (19.400). Se mejoraría ordenando o particionando la vista por país (*attribute clustering*).

4.4 Cómo reproducir

```
cd 03-base-datos/demo
./ejecutar-demo.sh                # Linux / macOS / Git Bash
.\ejecutar-demo.ps1               # Windows PowerShell
ORDERS=10000000 CUSTOMERS=500000 ./ejecutar-demo.sh   # escala del enunciado
MANTENER=1 ./ejecutar-demo.sh     # conservar datos y contenedor para explorar
```

El script levanta el servicio `oracle` del compose del backend, crea el usuario `demo_perf`, genera los datos, ejecuta los experimentos y guarda la salida en `demo/resultados/`. Al terminar elimina el usuario y detiene el contenedor. Scripts SQL en `demo/sql/`.

La versión `.sh` se ejecutó completa a escala 2M/100k (los resultados de esta sección). La `.ps1` se probó a escala reducida (100k/5k) y terminó sin errores.